

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: Laiton Tiboche	29/08/2026
	Nombre: Jerilin Meliza	

Análisis del programa base y detección de riesgos

La descarga del programa MARS versión 4.5 fue sencilla, ya que el programa Java necesario para su funcionamiento ya se encontraba instalado. Al abrir MARS, se realizaron las pruebas necesarias y se logró ejecutar el programa base de manera satisfactoria.

Posteriormente, se comenzó a analizar el código con el objetivo de identificar los posibles riesgos de datos (hazards) presentes en el pipeline.

Primero, se encontró una dependencia entre las instrucciones:

```
lw $t6, 0($t5)
mul $t7, $t6, $t0
```

La instrucción `lw $t6, 0($t5)` carga un dato desde memoria en el registro `$t6`, mientras que la siguiente instrucción, `mul $t7, $t6, $t0`, utiliza inmediatamente el valor almacenado en dicho registro. Esto genera una dependencia conocida como Load-Use Hazard, debido a que el dato cargado por `lw` puede no estar disponible todavía cuando la instrucción `mul` intenta utilizarlo.

Segundo, se identificó una dependencia entre:

```
mul $t7, $t6, $t0
addu $t8, $t7, $t1
```

La instrucción `mul` produce un resultado que se almacena en `$t7`, y la instrucción `addu` utiliza inmediatamente ese resultado. Esto genera una dependencia RAW (Read After Write), ya que `addu` necesita leer un dato que acaba de ser escrito por `mul`.

Tercero, se encontró otra dependencia entre:

```
addu $t8, $t7, $t1
sw $t8, 0($t9)
```

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: Laiton Tiboche	29/08/2026
	Nombre: Jerilin Meliza	

En este caso, addu genera el valor que se almacena en \$t8, mientras que sw necesita utilizar inmediatamente ese valor para escribirlo en memoria. Por lo tanto, también se presenta una dependencia RAW (Read After Write).

Cuarto se identificó una dependencia relacionada con el control y la actualización del índice del bucle:

addi \$t3, \$t3, 1

beq \$t3, \$t2, fin

La instrucción addi modifica el valor almacenado en \$t3, y posteriormente beq necesita leer ese nuevo valor para determinar si debe finalizar el bucle. Esta dependencia se produce entre instrucciones consecutivas relacionadas con la actualización y comprobación del índice del bucle.

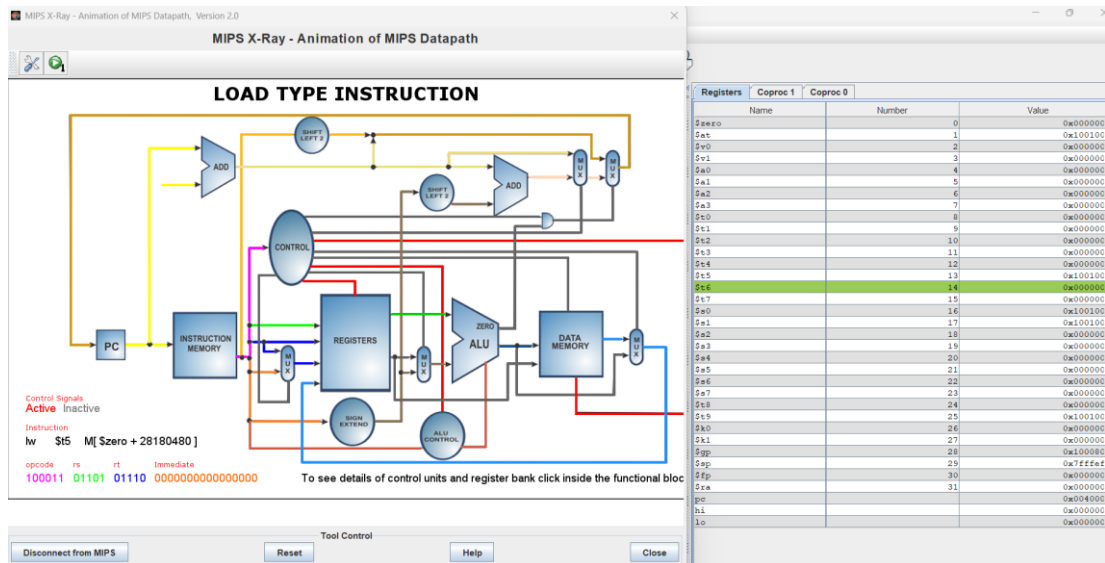
Resumen de los riesgos encontrados:

Instrucción Productora	Instrucción que usa el Dato	Registro	Tipo de dependencia
lw \$t6, 0(\$t5)	mul \$t7, \$t6, \$t0	\$t6	Load-Use
mul \$t7, \$t6, \$t0	addu \$t8, \$t7, \$t1	\$t7	RAW
addu \$t8, \$t7, \$t1	sw \$t8, 0(\$t9)	\$t8	RAW
addi \$t3, \$t3, 1	beq \$t3, \$t2, fin	\$t3	RAW entre iteraciones

Análisis mediante MIPS X-Ray

Para observar el proceso de ejecución de las instrucciones y analizar las dependencias dentro del pipeline, se utilizó la herramienta MIPS X-Ray. Esta herramienta permitió revisar el comportamiento de las instrucciones de manera individual e identificar los posibles riesgos de datos.

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: Laiton Tiboche	29/08/2026
	Nombre: Jerilin Meliza	



Optimización realizada

Para reducir los riesgos del pipeline, se reorganizaron algunas instrucciones independientes dentro del bucle.

Después de la instrucción: `lw $t6, 0($t5)`

se colocaron las siguientes instrucciones: `addu $t9, $s1, $t4` y `addi $t3, $t3, 1`

Estas instrucciones no dependen del valor que acaba de cargar `lw` en `$t6`. Por lo tanto, pueden ejecutarse mientras el dato cargado termina de estar disponible para su posterior utilización.

Con esta reorganización se busca reducir el Load-Use Hazard y disminuir las posibles paradas (stalls) generadas en el pipeline.

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: Laiton Tiboche	29/08/2026
	Nombre: Jerilin Meliza	

```

loop:
    # --- Condición de salida ---
    beq $t3, $t2, fin

    # --- Cálculo de dirección de memoria ---
    sll $t4, $t3, 2
    addu $t5, $s0, $t4

    # --- Carga de dato ---
    lw $t6, 0($t5)

    # --- Instrucciones independientes ---
    # Estas instrucciones no dependen del valor de $t3
    # Se colocan aquí para separar el lw de la o_i
    # y reducir el riesgo Load-Use en el pipeline
    addu $t9, $s1, $t4
    addi $t3, $t3, 1

    # --- Operación aritmética ---
    mul $t7, $t6, $t0
    addu $t8, $t7, $t1

    # --- Almacenamiento del resultado ---
    sw $t8, 0($t9)

```

Tabla optimización

Modificación	Antes	Después	Objetivo
Separación lw–mul	lw → mul	lw → addu → addi → mul	Reducir Load-Use Hazard
Cálculo de dirección de Y	Después de addu \$t8	Después de lw	Aprovechar una instrucción independiente
Incremento del índice	Antes de j loop	Antes de mul	Aprovechar un ciclo disponible
Operación mul	Inmediatamente después de lw	Después de instrucciones independientes	Reducir posible stall

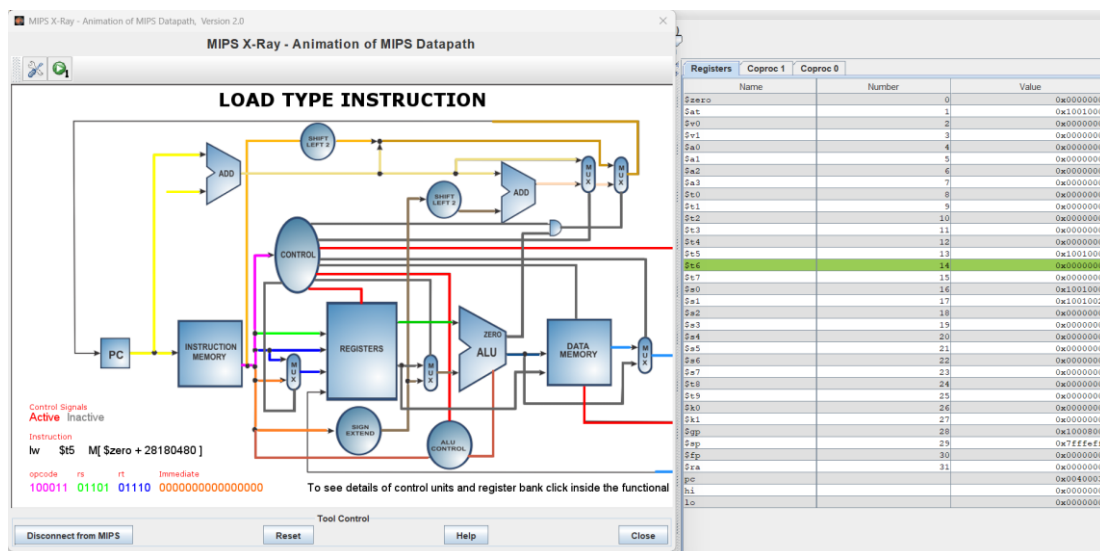
Ejecución del programa optimizado

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: Laiton Tiboche	29/08/2026
	Nombre: Jerilin Meliza	

Después de realizar la reorganización de las instrucciones, se volvió a ejecutar el programa utilizando el simulador de pipeline de MARS X-Ray.

La captura obtenida muestra el comportamiento del pipeline después de aplicar la optimización. Al comparar este resultado con el programa original, se puede observar que la reorganización permite separar algunas instrucciones productoras de sus instrucciones consumidoras.

El objetivo principal de esta modificación es aprovechar instrucciones que no tienen dependencia entre sí para ocupar los ciclos que podrían quedar como paradas del pipeline.



Conclusión

La optimización realizada no modifica el resultado del programa, sino que modifica el orden de determinadas instrucciones para aprovechar de una mejor manera los ciclos de ejecución del procesador.

Al colocar instrucciones independientes entre la carga lw y su posterior utilización en mul, se reduce la dependencia inmediata entre ambas instrucciones. Esto permite un uso más eficiente del pipeline y puede disminuir las paradas (stalls) ocasionadas por los riesgos de datos.

Asignatura	Datos del alumno	Fecha
Estructura de Computadores	Apellidos: Laiton Tiboche	29/08/2026
	Nombre: Jerilin Meliza	

Por medio de MIPS X-Ray fue posible comprobar visualmente el comportamiento de las instrucciones antes y después de la reorganización, permitiendo analizar el efecto de la optimización sobre el pipeline.